

МИНИСТЕРСТВО ЦИФРОВОГО РАЗВИТИЯ, СВЯЗИ И
МАССОВЫХ КОММУНИКАЦИЙ РОССИЙСКОЙ ФЕДЕРАЦИИ
САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
ТЕЛЕКОММУНИКАЦИЙ ИМ. ПРОФ. М.А. БОНЧ-БРУЕВИЧА
(СПбГУТ)

ФАКУЛЬТЕТ ИНФОКОММУНИКАЦИОННЫХ СЕТЕЙ И СИСТЕМ
КАФЕДРА СЕТЕЙ СВЯЗИ И ПЕРЕДАЧИ ДАННЫХ

ОТЧЁТ

по лабораторной работе № 7

«Кэширование и сжатие данных в Web»

Дисциплина: Разработка имитационных моделей инфокоммуникационных
сетей и систем

Руководитель, старший
преподаватель СС и ПД

подпись, дата

Дмитриева В. В.

Исполнители,
группа ИКПИ-33

подпись, дата

Морозов М.Е.

подпись, дата

Коломиец А.Р.

Санкт-Петербург

2026

1 Цель работы

Целью данной лабораторной работы является знакомство с технологией кэширования и сжатия данных, овладение навыками моделирования работы Web-приложений в среде Riverbed Modeler Academic Edition, а также анализ влияния кэширования и сжатия данных на работу Web-сервера и время загрузки Web-страниц.

2 Теоретические сведения

Кэширование данных — это механизм временного хранения копий часто запрашиваемых ресурсов (Web-страниц, изображений, скриптов) на промежуточных узлах сети или в локальном хранилище клиента с целью сокращения времени отклика и снижения нагрузки на сервер. При повторном обращении к ресурсу клиент или прокси-сервер может использовать кэшированную копию вместо обращения к исходному серверу, что значительно уменьшает задержку и объём передаваемых данных.

В протоколе HTTP механизмы кэширования реализуются с помощью заголовков Cache-Control, Expires, ETag и Last-Modified. Заголовок Cache-Control определяет политику кэширования: max-age задаёт время жизни кэшированного ресурса, no-cache требует обязательной перевалидации, а no-store запрещает сохранение данных в кэше. Условные запросы с использованием заголовков If-Modified-Since и If-None-Match позволяют клиенту проверить актуальность кэшированной копии без повторной загрузки ресурса.

Сжатие данных — это процесс уменьшения объёма передаваемой информации за счёт применения алгоритмов кодирования. В контексте Web-приложений наиболее распространёнными алгоритмами сжатия являются Gzip и Deflate, поддерживаемые большинством современных браузеров и серверов. Сжатие применяется к текстовым ресурсам (HTML, CSS, JavaScript, JSON) и позволяет уменьшить их объём на 60–90%, что существенно сокращает время загрузки страниц и снижает нагрузку на канал связи.

Механизм согласования сжатия реализуется через заголовки `Accept-Encoding` (в запросе клиента) и `Content-Encoding` (в ответе сервера). Клиент указывает поддерживаемые алгоритмы сжатия, а сервер выбирает оптимальный и применяет его к телу ответа. Сжатие увеличивает нагрузку на процессор сервера, однако при достаточных вычислительных ресурсах выигрыш в скорости передачи значительно превышает затраты на компрессию.

Совместное использование кэширования и сжатия данных обеспечивает максимальное снижение времени загрузки Web-страниц: кэширование устраняет необходимость повторной передачи ранее загруженных ресурсов, а сжатие уменьшает объём данных при первичной загрузке или обновлении кэша.

3 Ход выполнения работы

3.1 Создание нового проекта

В среде Riverbed Modeler Academic Edition создан новый проект. В меню File выбран пункт `New... → Project`. Проекту присвоено наименование в формате `<номер_студенческого>_Web`, первый сценарий назван `No_Cache_No_Compression`. В мастере Startup Wizard выбраны параметры: `Create empty scenario`, масштаб сети — `Campus`.

3.2 Создание сети

В окне Object Palette выбрана категория `internet_toolbox`. На рабочую область добавлены следующие объекты: `Application Config` (1 шт.), `Profile Config` (1 шт.), `HTTP Config` (1 шт.), `ethernet_wkstn` (клиентские рабочие станции), `ethernet_server` (1 шт. — Web-сервер), маршрутизаторы и коммутаторы для организации сетевой топологии.

Рабочие станции соединены с коммутаторами линиями `10Base_T`, маршрутизаторы — линией соответствующей пропускной способности. Узлы переименованы в соответствии с их функциональным назначением.

3.3 Настройка приложений и профилей

В объекте Application Config определено Web-приложение с параметрами HTTP-трафика: задан размер страниц, количество объектов на странице, интервал между запросами. В объекте Profile Config создан профиль пользователя, определяющий паттерн обращения клиентов к Web-серверу.

В объекте HTTP Config настроены параметры кэширования и сжатия для различных сценариев: в базовом сценарии No_Cache_No_Compression оба механизма отключены.

3.4 Создание сценариев

Созданы четыре сценария моделирования путём дублирования базового: No_Cache_No_Compression (без кэша и без сжатия), Cache_Only (с кэшированием, без сжатия), Compression_Only (без кэша, со сжатием), Cache_And_Compression (с кэшированием и сжатием).

В каждом сценарии соответствующим образом настроены параметры HTTP: включено или отключено кэширование на стороне клиента, включено или отключено сжатие на стороне сервера.

3.5 Сбор статистики

Через меню Choose Individual DES Statistics → Global Statistics выбраны следующие метрики: Client Http Page Response Time (seconds) — среднее время ответа на HTTP-запросы клиента; Server Http Load (requests/sec) — нагрузка на Web-сервер в запросах в секунду.

3.6 Запуск моделирования

Для всех четырёх сценариев выполнено имитационное моделирование. По окончании проект сохранён.

4 Результаты моделирования

4.1 Сравнение сценариев без кэша и с кэшем

На рисунке 1 представлен график среднего времени ответа на HTTP-запросы клиента (Client Http Page Response Time) для сценариев без кэша и с кэшем. На графике видно, что использование кэша в сети сильно уменьшает среднее время ожидания ответа, тогда как компрессия уменьшает его в меньшей степени.

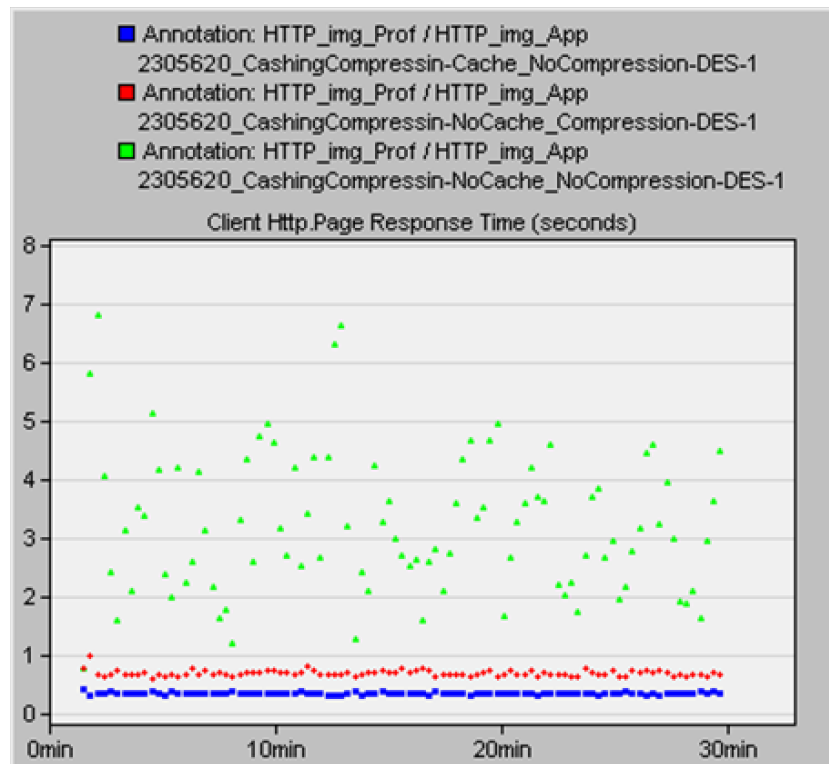


Рисунок 1 — Client Http Page Response Time (seconds)

На рисунке 2 представлен график нагрузки на Web-сервер (Server Http Load) для тех же сценариев. На графике видно, что компрессия увеличивает скорость обработки запросов, а использование кэша снижает нагрузку на сервер, поскольку часть запросов обслуживается из кэша.

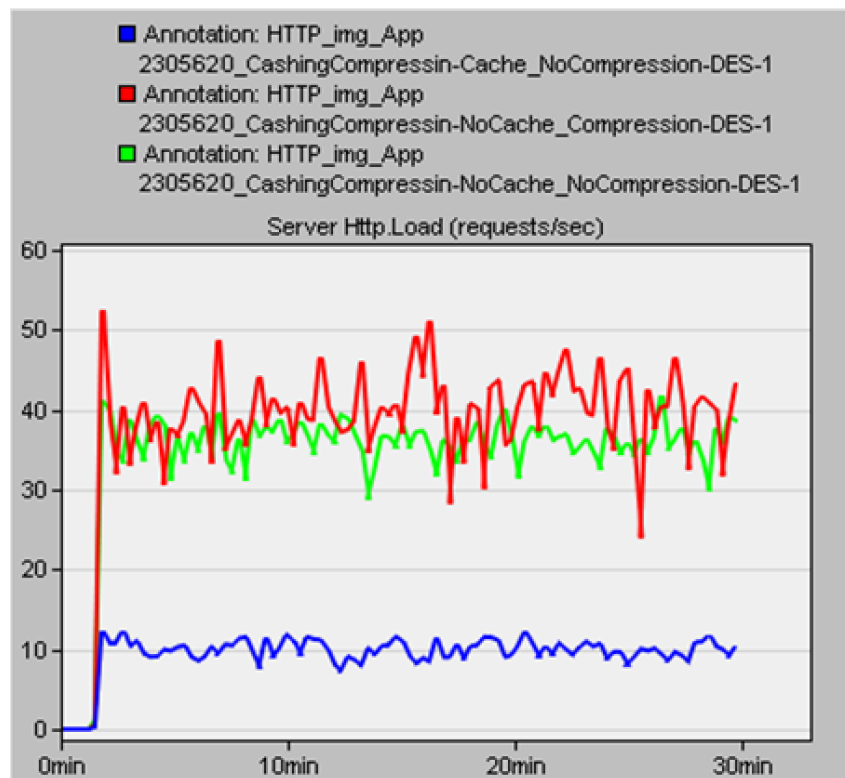


Рисунок 2 — Server Http Load (requests/sec)

4.2 Сравнение всех четырёх сценариев

На рисунке 3 представлен график среднего времени ответа на HTTP-запросы клиента для всех четырёх сценариев. На графике видно, что использование и кэша, и компрессии одновременно обеспечивает наилучшее снижение времени ожидания, однако этот показатель находится примерно на одном уровне с сетью, использующей только кэш.

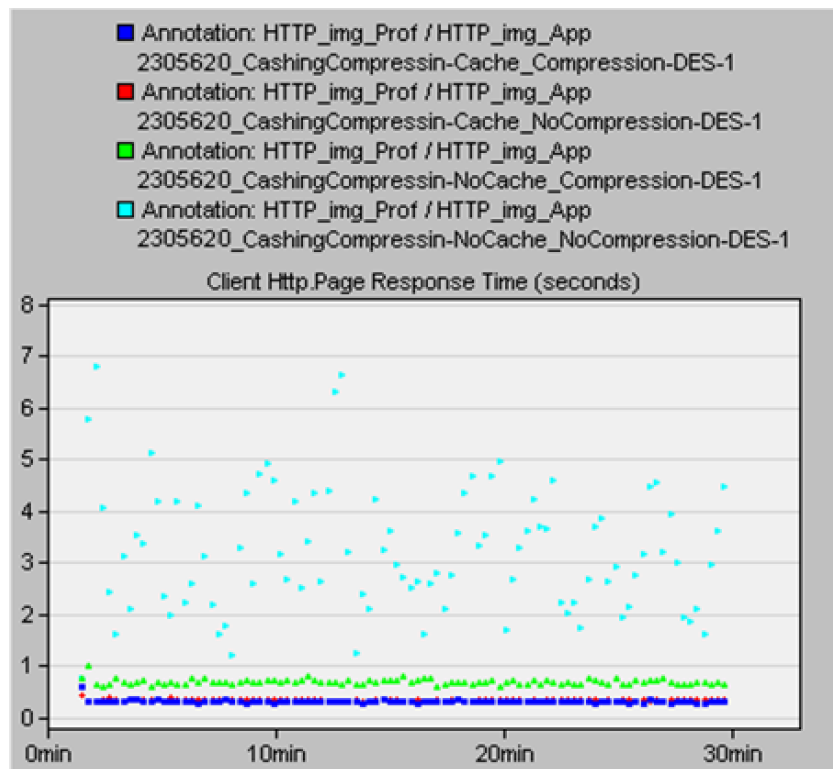


Рисунок 3 — Client Http Page Response Time (seconds)

На рисунке 4 представлен график нагрузки на Web-сервер для всех четырёх сценариев. На графике видно, что совместное использование кэша и компрессии не приводит к значительному повышению нагрузки на сервер по сравнению с использованием только кэша.

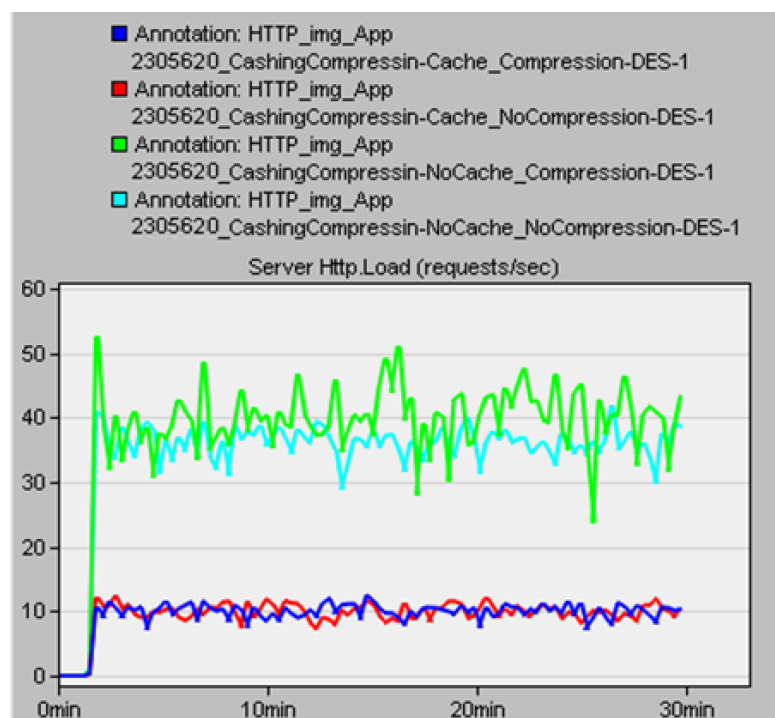


Рисунок 4 — Server Http Load (requests/sec)

5 Выводы

В ходе выполнения лабораторной работы получены следующие результаты.

Во-первых, кэширование данных оказывает наибольшее влияние на снижение среднего времени ответа на HTTP-запросы клиента. При включённом кэшировании время загрузки Web-страниц значительно сокращается, поскольку повторные запросы к ранее загруженным ресурсам обслуживаются из локального кэша без обращения к серверу.

Во-вторых, сжатие данных позволяет уменьшить объём передаваемого трафика, что снижает время загрузки страниц при первичном обращении, однако его эффект на время отклика менее выражен по сравнению с кэшированием. При этом сжатие увеличивает вычислительную нагрузку на сервер.

В-третьих, совместное применение кэширования и сжатия обеспечивает наилучшие показатели времени отклика. Однако выигрыш от добавления сжатия к уже кэшированной системе невелик, что объясняется тем, что основная доля экономии времени достигается за счёт кэширования.

Таким образом, результаты моделирования подтверждают целесообразность использования обоих механизмов — кэширования и сжатия — для оптимизации работы Web-приложений. Кэширование является первоочередной мерой для снижения времени загрузки, а сжатие — дополнительным средством уменьшения объёма трафика, особенно эффективным при первичных обращениях к серверу.